

MVLU COLLEGE
A.I. Practical No.9

- Aim :- A) Implement the Breadth First Search algorithm to solve a given problem.
B) Implement the Iterative Depth First Search algorithm to solve the same problem.
C) Compare the performance and efficiency of both algorithms.

Code:-

```
Practical 1.py - D:\Haripr\A\Practical 1.py (3.13.5)
File Edit Format Run Options Window Help
from collections import deque
print("Hariprasad Vishwakarma T127")
# Labyrinth layout mapping
labyrinth = {
    'Start': ['N1', 'N2'],
    'N1': ['Start', 'DeadEnd1', 'DeadEnd2'],
    'N2': ['Start', 'N3'],
    'DeadEnd1': ['N1'],
    'DeadEnd2': ['N1'],
    'N3': ['N2', 'N4', 'DeadEnd3'],
    'N4': ['N3', 'Goal'],
    'DeadEnd3': ['N3'],
    'Goal': ['N4']
}

# =====
# Breadth-First Search (BFS)
# =====
def bfs_locate_target(grid, start_node, goal_node="Goal"):
    frontier = deque([start_node])
    visited = set([start_node])

    print("BFS Exploration Order:")
    while frontier:
        active_path = frontier.popleft()
        curr = active_path[-1]
        print(f"Exploring node: {curr}")
        if curr == goal_node:
            return active_path

        for neighbor in grid.get(curr, []):
            if neighbor not in visited:
                visited.add(neighbor)
                new_path = list(active_path)
                new_path.append(neighbor)
                frontier.append(new_path)

    return None

# Execute BFS
bfs_path = bfs_locate_target(labyrinth, 'Start')
print(f"\nBFS Result Path: {bfs_path}\n")
```

MVLU COLLEGE

A.I. Practical No.9

```
Practical 1.py - D:\Hari\AI\Practical 1.py (3.13.5)
File Edit Format Run Options Window Help

    visited.add(neighbor)
    new_path = list(active_path)
    new_path.append(neighbor)
    frontier.append(new_path)

    return None

# Execute BFS
bfs_path = bfs_locate_target(labyrinth, 'Start')
print(f"\nBFS Result Path: {bfs_path}\n")

# =====
# Iterative Deepening Depth-First Search (IDDFS)
# =====
def depth_limited_search(grid, curr, goal_node, depth_rem, active_path):
    if curr == goal_node:
        return active_path
    if depth_rem <= 0:
        return None

    for neighbor in grid.get(curr, []):
        if neighbor not in active_path:
            new_path = list(active_path)
            new_path.append(neighbor)
            res = depth_limited_search(grid, neighbor, goal_node, depth_rem - 1, new_path)
            if res is not None:
                return res

    return None

def iddfs_locate_target(grid, start_node, goal_node="Goal", max_depth=10):
    print("IDDFS Iterations:")
    for limit in range(max_depth):
        print(f"Searching with depth limit: {limit}")
        res = depth_limited_search(grid, start_node, goal_node, limit, [start_node])
        if res is not None:
            return res

    return "No path found within max depth limit"

# Execute IDDFS
iddfs_path = iddfs_locate_target(labyrinth, 'Start')
print(f"\nIDDFS Result Path: {iddfs_path}")
```

Output:-

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
----- RESTART: D:\Hari\AI\Practical 1.py -----
Hariprasad Vishwakarma T127
BFS Exploration Order:
Exploring node: Start
Exploring node: N1
Exploring node: N2
Exploring node: DeadEnd1
Exploring node: DeadEnd2
Exploring node: N3
Exploring node: N4
Exploring node: DeadEnd3
Exploring node: Goal

BFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']

IDDFS Iterations:
Searching with depth limit: 0
Searching with depth limit: 1
Searching with depth limit: 2
Searching with depth limit: 3
Searching with depth limit: 4

IDDFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']
>>>
```

Hariprasad Vishwakarma
T127
TYCS